ServiX

Gestión de clientes, servicios y atención con IA

DESCRIPCIÓN DE ARQUITECTURA Y DISEÑO

Arquitectura y Diseño de ServiX

Vistas arquitectónicas, decisiones de diseño y calidad de la aplicación Android ServiX

Proyecto	ServiX — Gestión de clientes, servicios y atención con IA
Código del documento	DOC-02
Versión	1.0
Fecha	01/11/2026
Estándar de referencia	IEEE 1016-2009 e ISO/IEC/IEEE 42010:2022
Institución	Universidad de San Buenaventura
Asignatura	Optativa II
Docente	Ing. Jairo Armando Salcedo Aranda

Equipo de trabajo

Nicolas Quiroz Gomez	Product Owner
Juan Sebastian Garzon Suarez	Scrum Master
Juan Esteban Olaya Muñoz	Developer

Control del documento

Información general

Título	Arquitectura y Diseño de ServiX
Código	DOC-02
Versión	1.0
Fecha de emisión	01/11/2026
Estado	Versión 1.0 emitida; aprobaciones pendientes
Proyecto	ServiX
Autor	Equipo ServiX
Clasificación	Uso académico; sin información confidencial

Historial de revisiones

Versión	Fecha	Descripción del cambio	Autor
0.9	01/11/2026	Borrador inicial en formato plano (archivado como respaldo v1).	Equipo ServiX
1.0	01/11/2026	Rediseño completo: portada, control de versiones, numeración, figuras y tablas rotuladas, mayor profundidad.	Equipo ServiX

Aprobaciones

Rol	Nombre	Fecha	Firma / estado
Elaboró	Juan Esteban Olaya Muñoz (Developer)	Pendiente	Pendiente
Revisó	Juan Sebastian Garzon Suarez (Scrum Master)	Pendiente	Pendiente
Aprobó	Nicolas Quiroz Gomez (Product Owner)	Pendiente	Pendiente

Las aprobaciones se encuentran **pendientes**: no se registran firmas ni fechas de aprobación hasta que los responsables las emitan.

Tabla de contenido

1 Introducción . . . **6**
1.1 Propósito y alcance . . . 6
1.2 Resumen arquitectónico . . . 6
2 Partes interesadas y preocupaciones . . . **7**
2.1 Puntos de vista adoptados . . . 7
3 Vistas arquitectónicas . . . **8**
3.1 Vista de contexto . . . 8
3.2 Vista lógica: capas . . . 9
3.3 Vista de componentes . . . 10
3.4 Vista de información . . . 11
3.5 Vista de procesos . . . 11
3.5.1 Inicio de sesión . . . 12
3.5.2 Operaciones de negocio . . . 13
3.5.3 Chat con IA y escalamiento . . . 14
3.6 Vista de despliegue . . . 15
3.7 Vista de seguridad . . . 16
4 Decisiones arquitectónicas (ADR) . . . **18**
4.1 ADR-01: Firestore multiusuario por uid . . . 18
4.2 ADR-02: Firebase AI Logic en lugar de una clave en el APK . . . 18
4.3 ADR-03: Inyección de dependencias manual con AppContainer . . . 18
4.4 ADR-04: Roles inmutables protegidos por reglas . . . 19
4.5 ADR-05: Repositorios simulados para pruebas y degradación . . . 19
4.6 ADR-06: Casos de uso como capa de reglas de negocio . . . 19
4.7 ADR-07: Clasificación de intención por palabras clave . . . 20
4.8 ADR-08: Eventos a Make mediante webhook opcional . . . 20
5 Diseño detallado . . . **21**
5.1 Núcleo e inyección de dependencias . . . 21
5.2 Dominio . . . 21
5.3 Casos de uso del dominio . . . 22
5.4 Datos . . . 23
5.5 Presentación . . . 23
6 Manejo de errores y resiliencia . . . **25**
7 Persistencia y reglas de seguridad . . . **26**
8 Integración con IA y con Make . . . **27**
8.1 Asistente de IA . . . 27
8.2 Make . . . 27
9 Calidad: atributos y tácticas . . . **28**
10 Riesgos técnicos y deuda técnica . . . **29**
11 Evolución futura . . . **30**
Anexo A. Trazabilidad de decisiones con los requisitos . . . **31**

Lista de figuras

Figura 1. Vista de contexto de ServiX 8
Figura 2. Arquitectura por capas de ServiX 9
Figura 3. Componentes principales y sus dependencias 10
Figura 4. Modelo de datos en Firestore 11
Figura 5. Secuencia de inicio de sesión con validación de rol 12
Figura 6. Secuencia de una operación de negocio (alta y cambio de estado de servicio) 13
Figura 7. Secuencia del chat con IA y del escalamiento a atención humana 14
Figura 8. Vista de despliegue 15
Figura 9. Flujo de integración continua en Azure Pipelines 16
Figura 10. Diagrama de clases del dominio y los repositorios 22

Lista de tablas

Tabla 1. Resumen de decisiones y atributos de la arquitectura . . . 6
Tabla 2. Partes interesadas y sus preocupaciones . . . 7
Tabla 3. Puntos de vista y vistas del documento . . . 7
Tabla 4. Elementos del contexto . . . 8
Tabla 5. Capas, paquetes y responsabilidades . . . 9
Tabla 6. Componentes principales . . . 10
Tabla 7. Nodos de despliegue . . . 15
Tabla 8. Controles de seguridad por ruta de Firestore . . . 16
Tabla 9. Registro ADR-01 . . . 18
Tabla 10. Registro ADR-02 . . . 18
Tabla 11. Registro ADR-03 . . . 18
Tabla 12. Registro ADR-04 . . . 19
Tabla 13. Registro ADR-05 . . . 19
Tabla 14. Registro ADR-06 . . . 19
Tabla 15. Registro ADR-07 . . . 20
Tabla 16. Registro ADR-08 . . . 20
Tabla 17. Clases del núcleo . . . 21
Tabla 18. Modelos de dominio . . . 21
Tabla 19. Casos de uso agrupados . . . 22
Tabla 20. Implementaciones de repositorio . . . 23
Tabla 21. Pantallas y componentes de presentación . . . 23
Tabla 22. Estrategias de manejo de errores . . . 25
Tabla 23. Parámetros del asistente . . . 27
Tabla 24. Atributos de calidad y tácticas aplicadas . . . 28
Tabla 25. Deuda técnica y riesgos de arquitectura . . . 29
Tabla 26. Evolución propuesta . . . 30
Tabla 27. Relación entre decisiones arquitectónicas y requisitos . . . 31

1 Introducción

1.1 Propósito y alcance

Este documento describe la arquitectura de ServiX y el diseño detallado de sus componentes más relevantes. Sigue la estructura de ISO/IEC/IEEE 42010 (partes interesadas, preocupaciones, puntos de vista y vistas) y el contenido de diseño de IEEE 1016. Se redactó a partir del código fuente del repositorio, por lo que describe el sistema **tal como está implementado**, incluyendo sus limitaciones y deuda técnica.

1.2 Resumen arquitectónico

ServiX es una aplicación Android de un solo módulo (app) organizada en cuatro capas: **presentación** (Compose y `MainViewModel`), **dominio** (modelos, interfaces de repositorio y casos de uso), **datos** (repositorios Firebase, IA, red y simulados) y **núcleo** (inyección de dependencias, navegación, tema y utilidades). La dependencia fluye hacia el dominio: la presentación y los datos conocen el dominio, y el dominio no conoce a ninguna de las dos.

Tabla 1. Resumen de decisiones y atributos de la arquitectura

Aspecto	Decisión vigente
Estilo	Arquitectura en capas con MVVM y casos de uso; repositorios definidos por interfaz.
Persistencia	Cloud Firestore multiusuario por `uid` con escuchas en tiempo real.
Autenticación	Firebase Authentication (correo y contraseña) con rol almacenado en Firestore.
Inteligencia artificial	Firebase AI Logic con `gemini-2.5-flash`; sin clave en el APK.
Inyección de dependencias	Manual, mediante el objeto `AppContainer`.
Integraciones	Webhook de Make (OkHttp); interfaces de Retell y n8n sin implementación.
Pruebas	JUnit, Robolectric y repositorios simulados; CI en Azure Pipelines.

2 Partes interesadas y preocupaciones

Tabla 2. Partes interesadas y sus preocupaciones

Parte interesada	Rol	Preocupaciones principales	Vistas que las atienden
Cliente	Usuario final	Solicitar servicios, ver su avance, recibir atención rápida, privacidad de sus datos.	Lógica, procesos, seguridad
Administrador	Usuario operador	Gestionar clientes y órdenes con fluidez, atender casos escalados, ver estadísticas.	Lógica, componentes, procesos
Product Owner	Scrum	Que las funciones del backlog estén implementadas y trazables.	Contexto, componentes
Scrum Master	Scrum	Calidad del proceso y del pipeline de integración continua.	Despliegue (CI/CD)
Developer	Equipo técnico	Mantenibilidad, claridad de capas, facilidad de prueba, tiempos de compilación.	Lógica, datos, componentes
Docente (Optativa II)	Evaluador	Cumplimiento de la guía, documentación y evidencias.	Todas
Propietario del proyecto Firebase	Operación	Costos, cuotas, claves y reglas de seguridad.	Seguridad, datos, despliegue

2.1 Puntos de vista adoptados

Tabla 3. Puntos de vista y vistas del documento

Punto de vista	Vista	Sección	Diagrama
Contexto	Sistema y entorno	3.1	Figura de contexto
Lógico	Capas y paquetes	3.2	Figura de arquitectura en capas
Componentes	Componentes y dependencias	3.3	Figura de componentes y de clases
Información	Modelo de datos	3.4	Figura entidad-relación
Procesos	Secuencias de interacción	3.5	Tres figuras de secuencia
Despliegue	Nodos y CI/CD	3.6	Figuras de despliegue y de CI/CD
Seguridad	Control de acceso y secretos	3.7	Tabla de reglas

3 Vistas arquitectónicas

3.1 Vista de contexto

El sistema interactúa con dos actores humanos y cuatro servicios externos. No existe un backend propio: todo el estado compartido vive en Firebase.

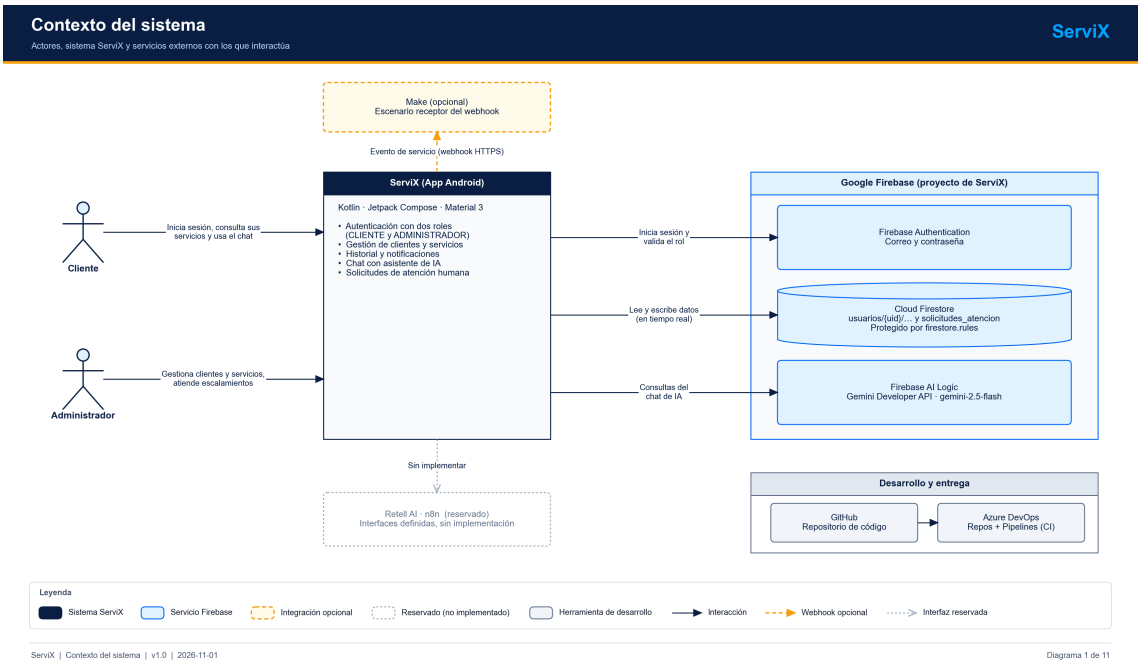


Figura 1. Vista de contexto de ServiX

Fuente: Diagrama del Equipo ServiX (04_Arquitectura_Diagramas).

Tabla 4. Elementos del contexto

Elemento	Tipo	Interacción
Cliente	Actor	Se registra, solicita servicios y conversa con el asistente.
Administrador	Actor	Gestiona clientes, servicios y solicitudes.
Firebase Authentication	Servicio externo	Valida credenciales y emite la identidad.
Cloud Firestore	Servicio externo	Almacena y sincroniza los datos.
Firebase AI Logic / Gemini	Servicio externo	Genera las respuestas del asistente.
Make	Servicio externo opcional	Recibe eventos de servicio por webhook.

3.2 Vista lógica: capas

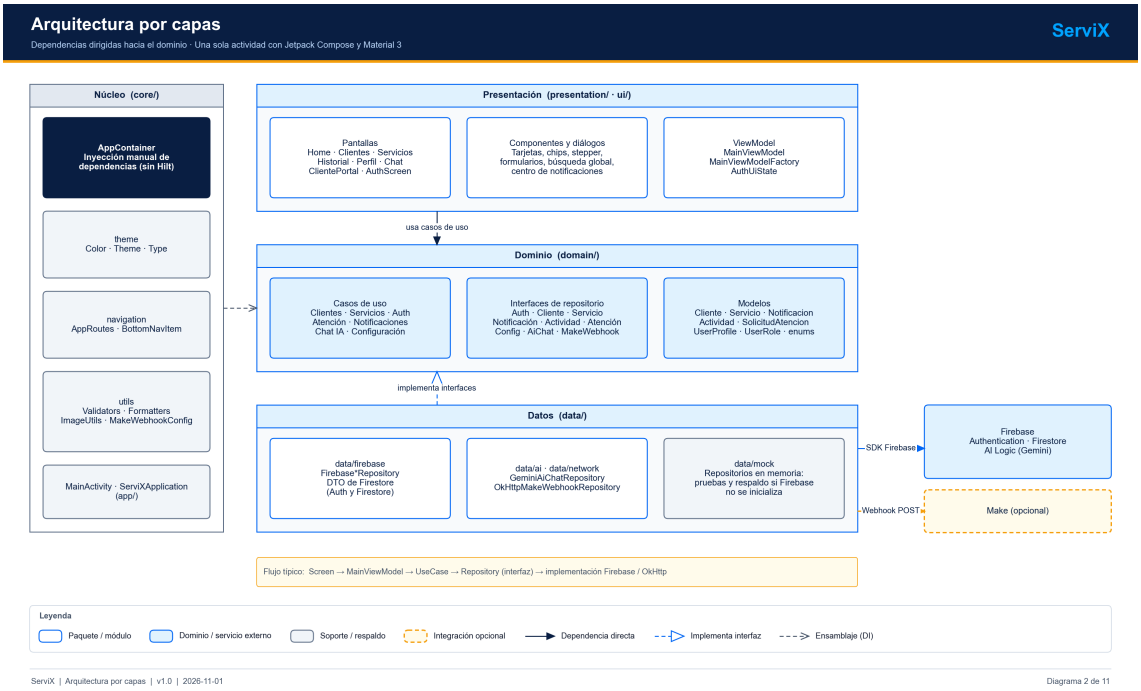


Figura 2. Arquitectura por capas de ServiX

Fuente: Diagrama del Equipo ServiX (04_Arquitectura_Diagramas).

Tabla 5. Capas, paquetes y responsabilidades

Capa	Paquete ( )	Responsabilidad
Presentación	`presentation.screens`, `components`, `dialogs`, `viewmodel`; `ui.screens.AuthScreen`; `app.MainActivity`	Pantallas Compose, estado de la interfaz (`MainViewModel`) y navegación.
Dominio	`domain.model`, `domain.repository`, `domain.usecase`	Reglas del negocio, entidades, contratos de repositorio y casos de uso.
Datos	`data.firebase`, `data.ai`, `data.network`, `data.mock`	Implementaciones de los repositorios: Firestore, Gemini, Make y simuladas.
Núcleo	`core.di`, `core.navigation`, `core.theme`, `core.utils`	Composición de dependencias, rutas, tema Material 3 y utilidades.

Las dependencias respetan la dirección hacia el dominio. La excepción aceptada es que `MainViewModel` obtiene sus colaboradores por defecto desde `AppContainer` (núcleo), lo que equivale a un localizador de servicios y se discute en el ADR-03.

3.3 Vista de componentes

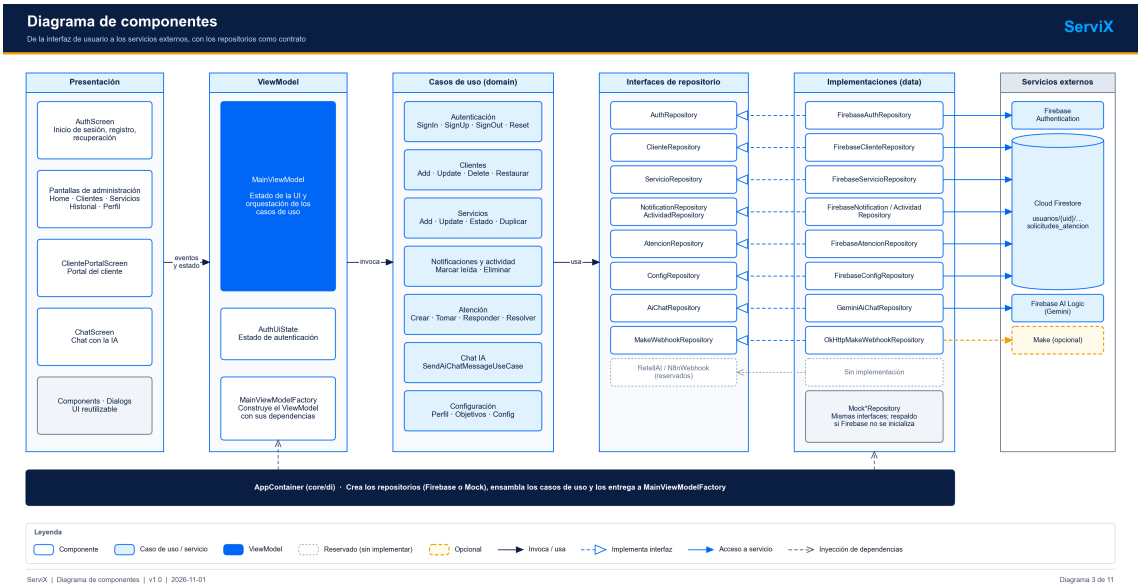


Figura 3. Componentes principales y sus dependencias

Fuente: Diagrama del Equipo ServiX (04_Arquitectura_Diagramas).

Tabla 6. Componentes principales

Componente	Responsabilidad	Colaboradores
`MainActivity` / `ServiXApp`	Punto de entrada; inicializa Firebase y el contenedor; decide la navegación por rol.	`AppContainer`, `MainViewModel`
`MainViewModel`	Expone flujos de estado y delega acciones en los casos de uso.	Repositorios y casos de uso
Casos de uso	Operaciones del negocio con validación y registro de actividad.	Repositorios del dominio
Repositorios Firebase	Escuchas y escrituras en Firestore bajo `usuarios/{uid}`.	Firestore, Auth
`GeminiAiChatRepository`	Llamada al modelo con historial y contexto.	Firebase AI Logic
`OkHttpMakeWebhookRepository`	Envío de eventos de servicio a Make.	OkHttp, `MakeWebhookConfig`
Repositorios mock	Datos en memoria para pruebas y degradación.	`MockDataStore`

3.4 Vista de información

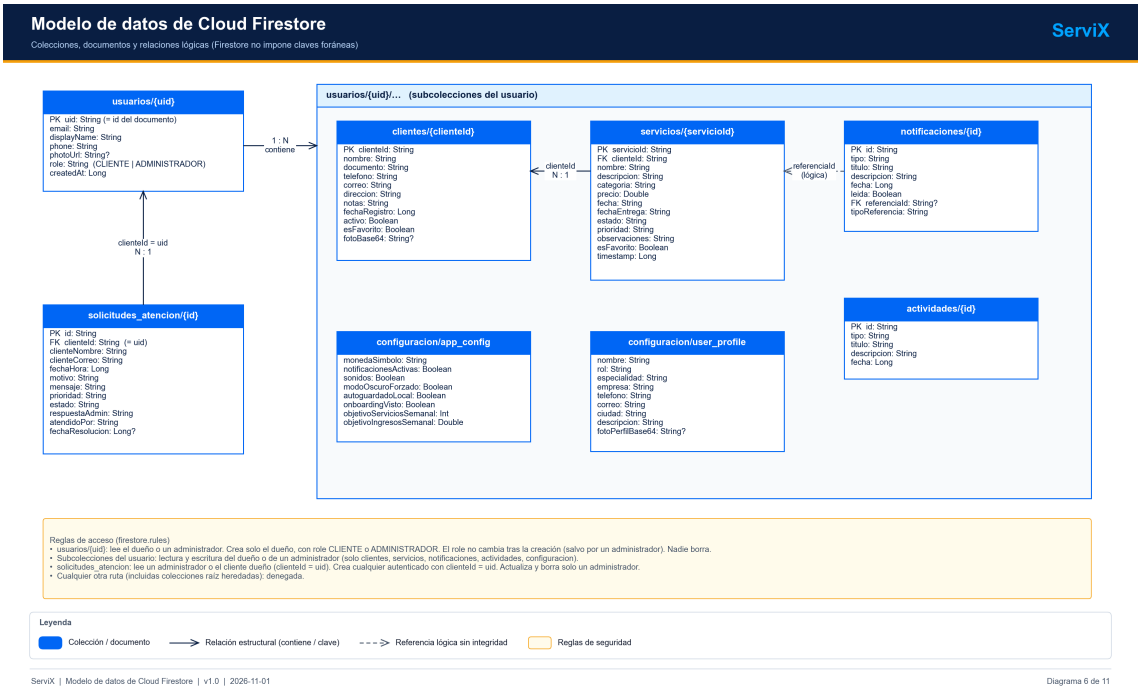


Figura 4. Modelo de datos en Firestore

Fuente: Diagrama del Equipo ServiX (04_Arquitectura_Diagramas).

El modelo se detalla en DOC-07. Desde el punto de vista arquitectónico, las decisiones relevantes son: (1) aislamiento por usuario mediante subcolecciones bajo `usuarios/{uid}`; (2) una colección raíz para solicitudes de atención, necesaria porque cliente y administrador deben leer los mismos registros; y (3) uso de DTOs con valores por defecto para tolerar documentos incompletos.

3.5 Vista de procesos

3.5.1 Inicio de sesión

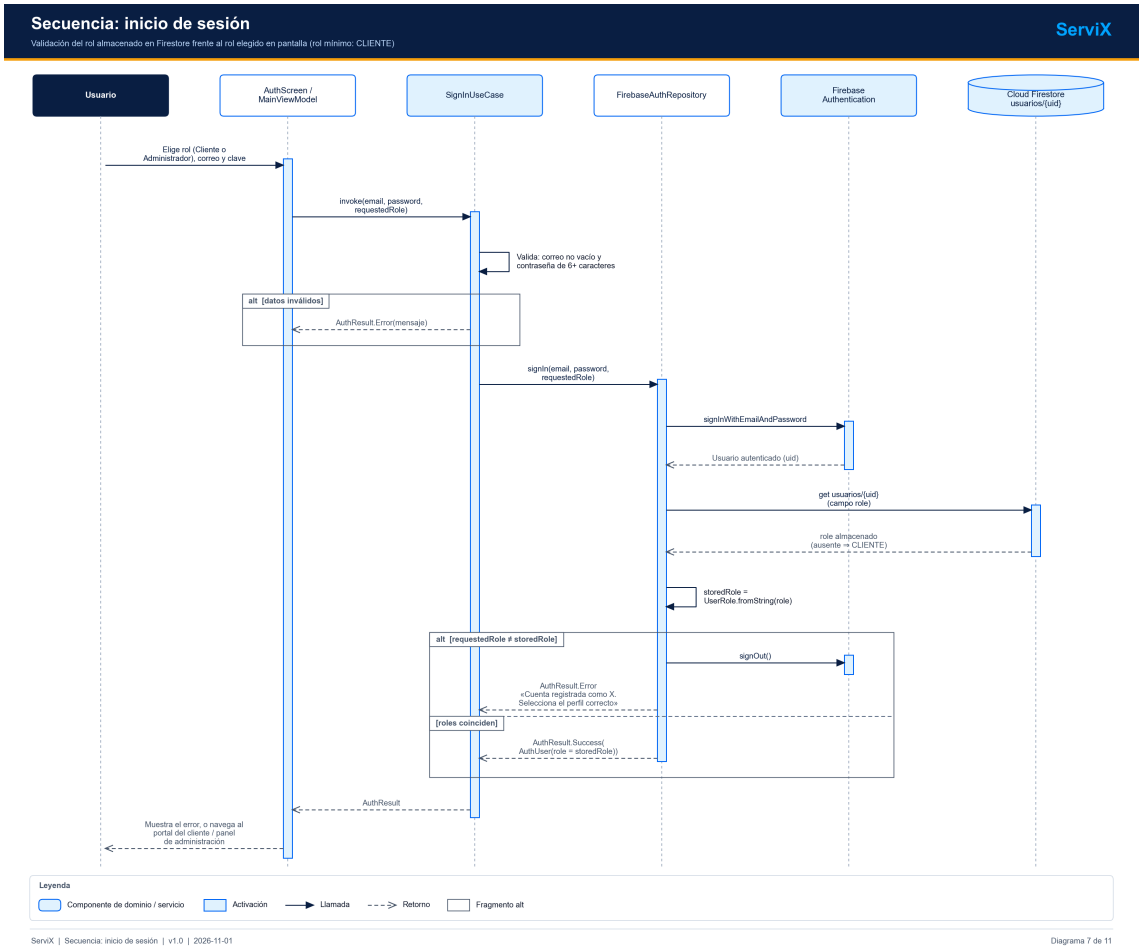


Figura 5. Secuencia de inicio de sesión con validación de rol

Fuente: Diagrama del Equipo ServiX (04_Arquitectura_Diagramas).

El sistema autentica con Firebase, lee el rol almacenado en `usuarios/{uid}` y lo compara con el rol elegido en pantalla. Si no coinciden, cierra la sesión y rechaza el acceso; el rol efectivo siempre es el almacenado. Si la lectura del rol falla, se asume `CLIENTE`.

3.5.2 Operaciones de negocio

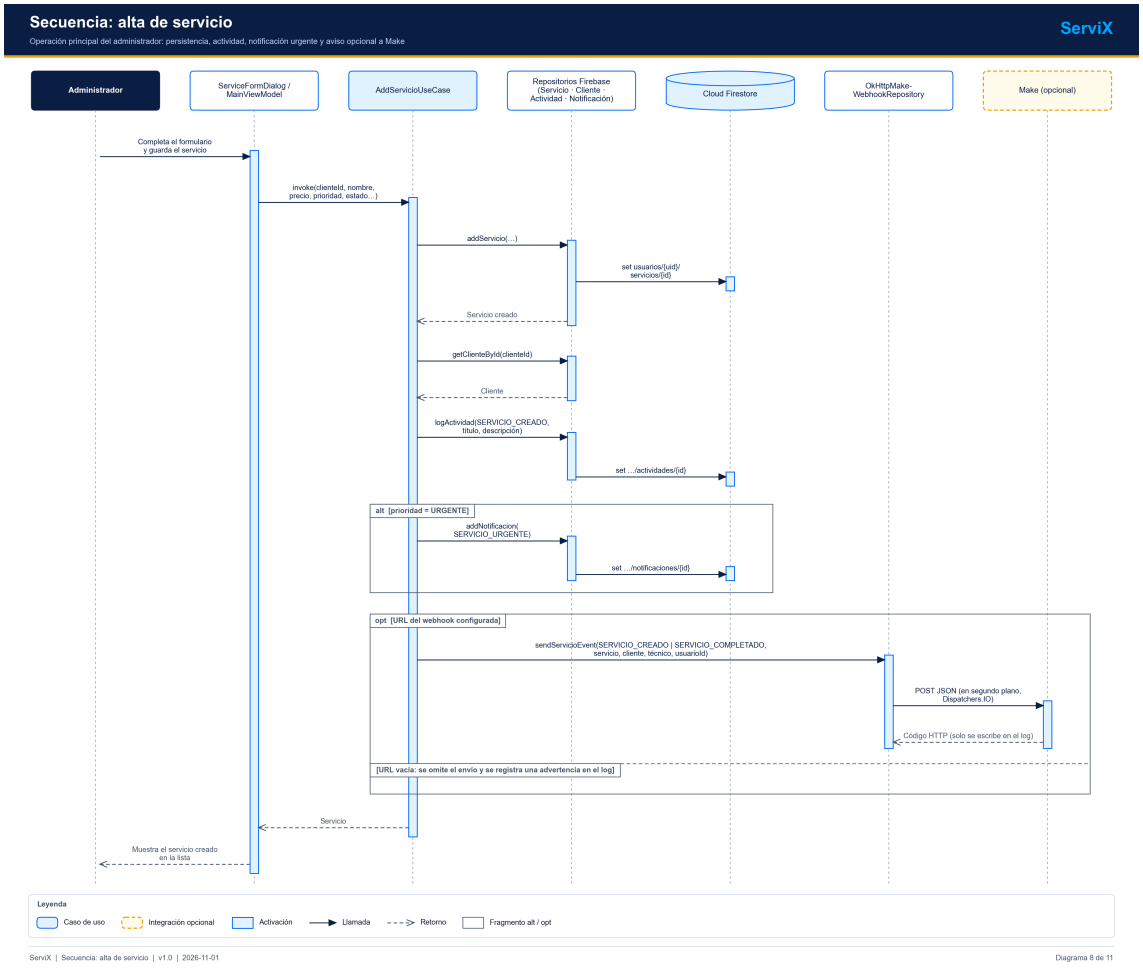


Figura 6. Secuencia de una operación de negocio (alta y cambio de estado de servicio)

Fuente: Diagrama del Equipo ServiX (04_Arquitectura_Diagramas).

La interfaz invoca un método de `MainViewModel`, que llama al caso de uso. El caso de uso valida, escribe mediante el repositorio, registra la actividad, genera notificaciones y, en los estados que corresponden, envía el evento a Make. Las escuchas de Firestore devuelven el cambio a la interfaz sin recarga explícita.

3.5.3 Chat con IA y escalamiento

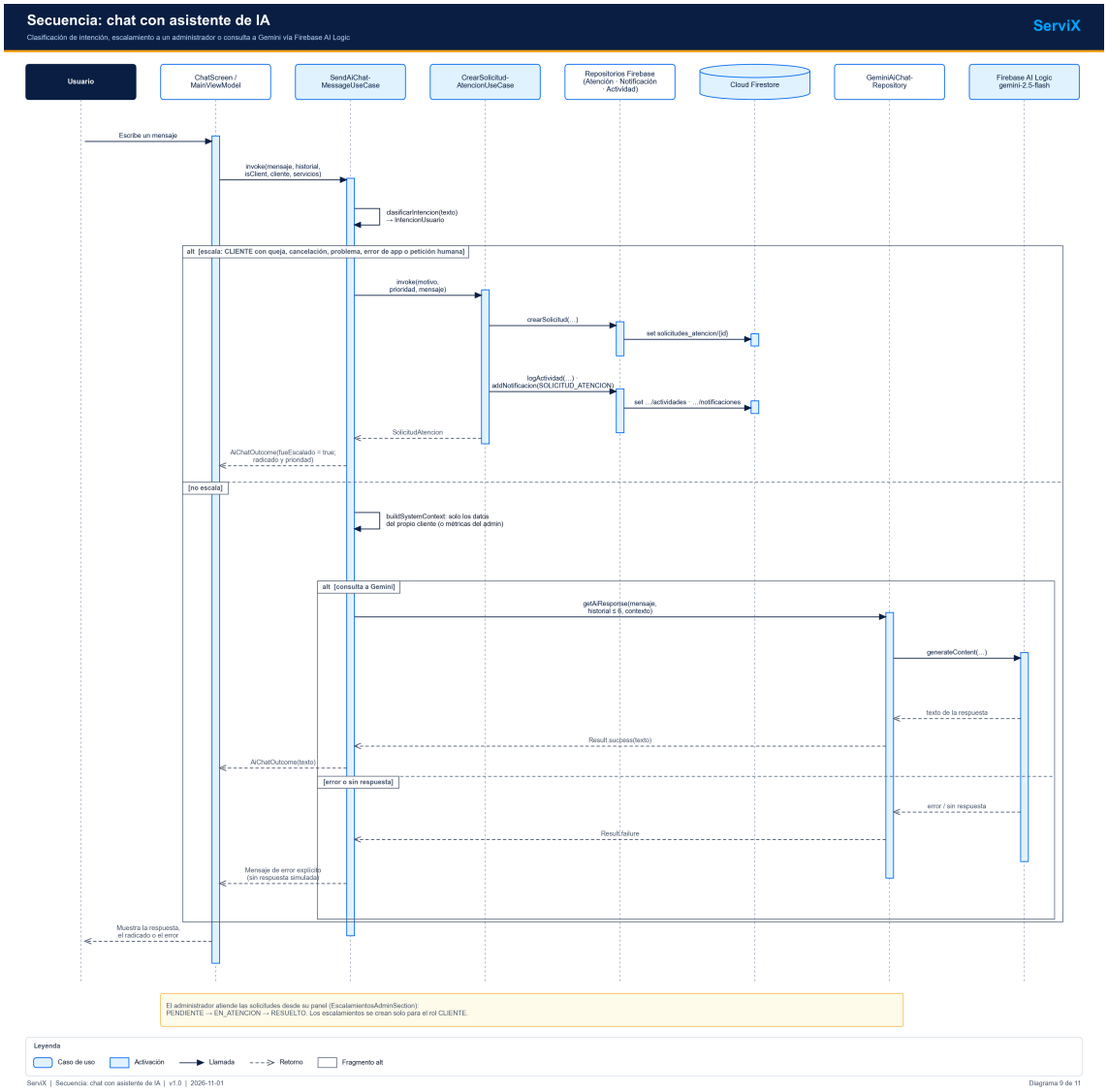


Figura 7. Secuencia del chat con IA y del escalamiento a atención humana

Fuente: Diagrama del Equipo ServiX (04_Arquitectura_Diagramas).

`SendAiChatMessageUseCase` clasifica el mensaje por palabras clave. Si es un cliente y la intención exige escalamiento, crea la solicitud sin consultar al modelo. En caso contrario construye el contexto del sistema y llama a `GeminiAiChatRepository`.

3.6 Vista de despliegue

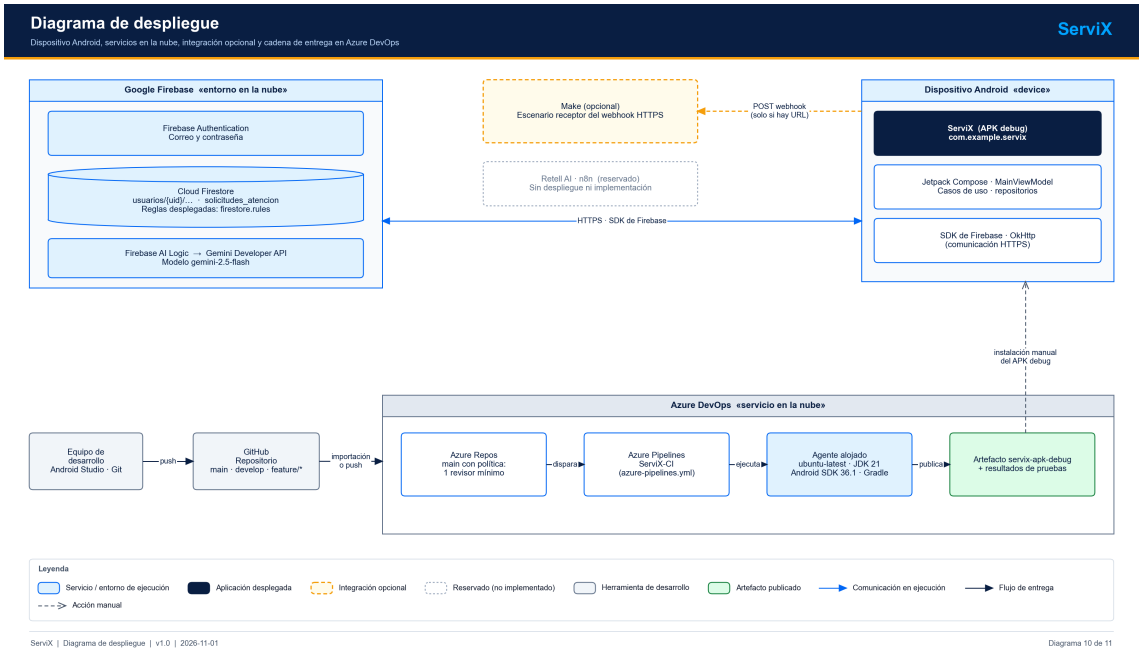


Figura 8. Vista de despliegue

Fuente: Diagrama del Equipo ServiX (04_Arquitectura_Diagramas).

Tabla 7. Nodos de despliegue

Nodo	Contenido	Observación
Dispositivo Android	APK de ServiX (minSdk 26, targetSdk 36).	Instalación manual o desde el artefacto del pipeline.
Proyecto Firebase	Authentication, Firestore con reglas desplegadas y AI Logic.	Reglas versionadas en `firestore.rules`.
Make (opcional)	Escenario con webhook.	Configuración fuera del repositorio.
Azure Pipelines	Agente `ubuntu-latest` con JDK 21 y SDK 36.1.	Compila, prueba y publica `servix-apk-debug`.

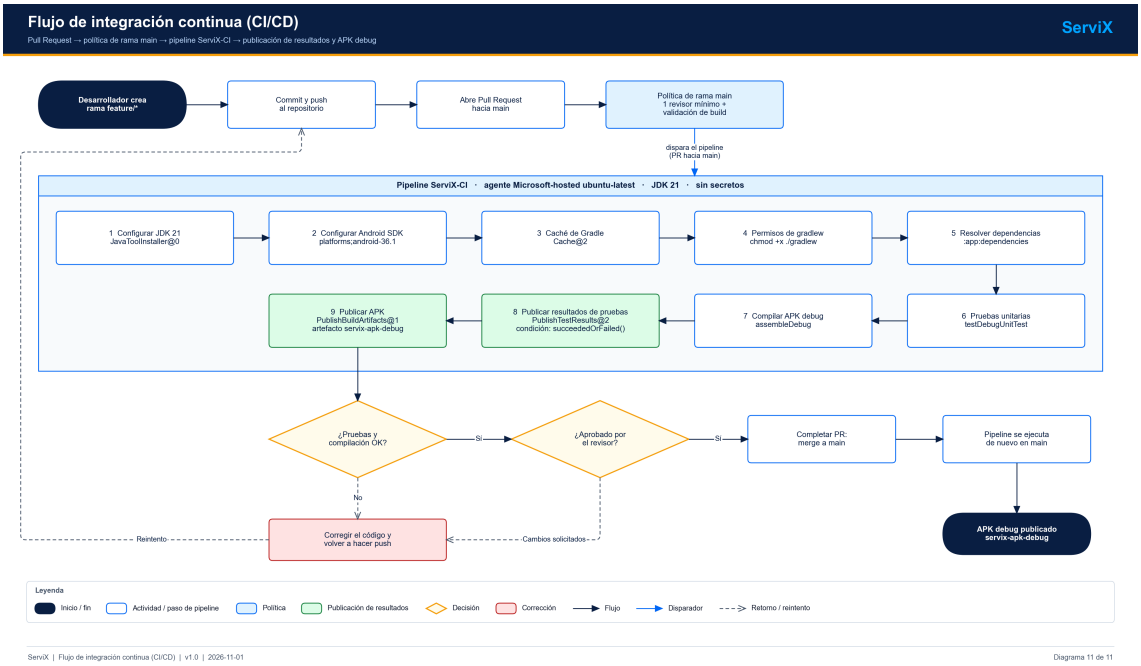


Figura 9. Flujo de integración continua en Azure Pipelines

Fuente: Diagrama del Equipo ServiX (04_Arquitectura_Diagramas).

El pipeline `ServiX-CI` se dispara con cambios en `main` y con Pull Requests hacia `main`. Instala JDK 21, configura el SDK de Android, restaura la caché de Gradle, resuelve dependencias de `:app`, ejecuta `testDebugUnitTest`, compila `assembleDebug`, publica los resultados de pruebas y publica el APK. No hay despliegue continuo: el APK de depuración es el único artefacto.

3.7 Vista de seguridad

La seguridad descansa en tres mecanismos: autenticación (Firebase Authentication), autorización en el servidor (reglas de Firestore) y ausencia de secretos en el cliente y en el repositorio. La interfaz oculta funciones por rol, pero no es una barrera de seguridad.

Tabla 8. Controles de seguridad por ruta de Firestore

Ruta	Lectura	Escritura	Notas
`usuarios/{userId}`	Dueño o administrador	Creación: dueño con rol válido. Actualización: dueño sin cambiar el rol, o administrador. Borrado: nadie.	Rol inmutable para el dueño.
`usuarios/{userId}/{sub}/{doc}`	Dueño o administrador	Dueño o administrador	Solo `clientes`, `servicios`, `notificaciones`, `actividades` y `configuracion`.
`solicitudes_atencion/{id}`	Administrador, o cliente dueño de la solicitud	Creación: usuario autenticado a su nombre con límites de longitud. Actualización y borrado: administrador.	Evita que un cliente lea solicitudes ajenas.

Ruta	Lectura	Escritura	Notas
Cualquier otra ruta	Denegada	Denegada	Regla final `if false`.

4 Decisiones arquitectónicas (ADR)

Cada decisión se documenta con su contexto, la decisión tomada, las alternativas consideradas y las consecuencias. El estado de todas es **Aceptada**, salvo que se indique lo contrario.

4.1 ADR-01: Firestore multiusuario por uid

Tabla 9. Registro ADR-01

Campo	Contenido
Contexto	La aplicación guarda datos de negocio por usuario y debe impedir que un usuario lea los de otro. No hay backend propio.
Decisión	Almacenar clientes, servicios, notificaciones, actividades y configuración bajo `usuarios/{uid}` y proteger la ruta con reglas que comparan `request.auth.uid`.
Alternativas consideradas	Colecciones raíz con un campo `ownerId`; una base de datos relacional con API propia.
Consecuencias	Aislamiento simple y reglas legibles. Contrapartida: los datos de un cliente y de un administrador viven en árboles distintos; el flujo en que un administrador ve datos de un cliente requiere leer otro árbol y no está implementado en la interfaz.

4.2 ADR-02: Firebase AI Logic en lugar de una clave en el APK

Tabla 10. Registro ADR-02

Campo	Contenido
Contexto	La primera versión empaquetaba una clave de Gemini en `BuildConfig` y la enviaba en la URL (hallazgo H-02).
Decisión	Usar el SDK `firebase-ai` con `GenerativeBackend.googleAI()`; la credencial vive en el proyecto Firebase.
Alternativas consideradas	Proxy propio con Cloud Functions; mantener la clave en el APK con restricciones.
Consecuencias	Sin clave extraíble del APK. Pendiente: registrar App Check para limitar el uso a la aplicación legítima y configurar cuotas.

4.3 ADR-03: Inyección de dependencias manual con AppContainer

Tabla 11. Registro ADR-03

Campo	Contenido
Contexto	El proyecto es pequeño y el equipo necesita una composición explícita y rápida de compilar en equipos de 4 GB de RAM.
Decisión	Un objeto singleton `AppContainer` crea repositorios y casos de uso; `MainViewModelFactory` los entrega al ViewModel.
Alternativas consideradas	Hilt o Koin.

Campo	Contenido
Consecuencias	Menos dependencias y compilación más rápida. A cambio, `MainViewModel` usa `AppContainer` como valor por defecto (localizador de servicios) y las dependencias se resuelven en tiempo de ejecución.

4.4 ADR-04: Roles inmutables protegidos por reglas

Tabla 12. Registro ADR-04

Campo	Contenido
Contexto	El rol decide qué ve el usuario y qué puede modificar; no debe poder cambiarse desde el cliente.
Decisión	Guardar `role` en `usuarios/{uid}`, leerlo al iniciar sesión (nunca confiar en la selección de pantalla) y bloquear su modificación por el dueño en las reglas.
Alternativas consideradas	Custom claims de Firebase asignados desde el servidor.
Consecuencias	Rol coherente entre cliente y servidor. Limitación abierta: el registro permite elegir `ADMINISTRADOR` y las reglas lo aceptan (H-15). Los claims asignados por servidor serían la solución definitiva.

4.5 ADR-05: Repositorios simulados para pruebas y degradación

Tabla 13. Registro ADR-05

Campo	Contenido
Contexto	Las pruebas deben ejecutarse sin red ni Firebase y la aplicación no debe cerrarse si Firebase falla al iniciar.
Decisión	Cada repositorio tiene una implementación `Mock` en memoria; `AppContainer` activa los mock si la inicialización o la creación de un repositorio Firebase falla.
Alternativas consideradas	Emulador de Firebase en las pruebas; fallar de forma explícita.
Consecuencias	Pruebas rápidas y deterministas. Riesgo: la degradación es silenciosa y el usuario puede creer que sus datos se guardan (H-11, Bug 46).

4.6 ADR-06: Casos de uso como capa de reglas de negocio

Tabla 14. Registro ADR-06

Campo	Contenido
Contexto	Las operaciones combinan varias acciones: escribir, registrar actividad, notificar y avisar a Make.
Decisión	Un caso de uso por operación en `domain.usecase`, con repositorios inyectados por constructor.
Alternativas consideradas	Lógica directamente en el ViewModel.

Campo	Contenido
Consecuencias	Lógica probable en JVM sin Android. El `MainViewModel` sigue siendo grande por concentrar el estado de toda la aplicación.

4.7 ADR-07: Clasificación de intención por palabras clave

Tabla 15. Registro ADR-07

Campo	Contenido
Contexto	El escalamiento a una persona debe ser predecible y no depender de la disponibilidad del modelo.
Decisión	Clasificar el mensaje con reglas de texto antes de consultar al modelo; las intenciones críticas crean la solicitud sin invocar a la IA.
Alternativas consideradas	Clasificación con el propio modelo.
Consecuencias	Comportamiento determinista y comprobable. Limitación: las palabras clave no cubren todos los casos ni variaciones del lenguaje.

4.8 ADR-08: Eventos a Make mediante webhook opcional

Tabla 16. Registro ADR-08

Campo	Contenido
Contexto	El negocio puede querer avisos o reportes automáticos sin que el equipo mantenga un backend.
Decisión	Enviar un JSON por HTTP POST a una URL configurada al compilar (`local.properties` o `MAKE_WEBHOOK_URL`); si no existe, el envío se omite.
Alternativas consideradas	Cloud Functions; integración nativa con n8n o Retell.
Consecuencias	Integración desacoplada y sin secretos en el repositorio. La URL se compila en el APK y es extraíble de él.

5 Diseño detallado

5.1 Núcleo e inyección de dependencias

Tabla 17. Clases del núcleo

Clase	Responsabilidad	Detalles
`AppContainer`	Composición de repositorios y casos de uso.	`init` inicializa Firebase; propiedades perezosas con `try/catch` que degradan a mock; `useMockRepositories()` y `useFirebaseRepositories()` para pruebas.
`MainViewModelFactory`	Crea `MainViewModel` con sus dependencias.	Toma casos de uso del contenedor.
`AppRoutes`, `BottomNavItem`	Rutas y destinos de la barra inferior.	Seis destinos con `testTag`.
`MakeWebhookConfig`	Lee la URL compilada.	`BuildConfig.MAKE_WEBHOOK_URL`; `isConfigured` indica si está vacía.
`Validators`, `Formatters`, `ImageUtils`	Validación, formato de moneda/fecha y manejo de imágenes.	Utilidades puras.
Tema (`Color.kt`, `Theme.kt`, `Type.kt`)	Identidad visual Material 3.	Paleta azul de ServiX, modo claro y oscuro.

5.2 Dominio

Tabla 18. Modelos de dominio

Modelo	Atributos relevantes	Comportamiento
`AuthUser`	uid, correo, nombre, teléfono, foto, rol, fecha de creación.	Rol por defecto `CLIENTE`.
`UserRole`	`CLIENTE`, `ADMINISTRADOR`.	`fromString` acepta valores heredados y usa `CLIENTE` por defecto.
`Cliente`	Identificador, nombre, documento, contacto, notas, favorito, foto.	`iniciales` y `calcularSegmentos`.
`Servicio`	Cliente, nombre, categoría, precio, fechas, estado, prioridad, observaciones.	Estados `EstadoServicio` y prioridades `PrioridadServicio`.
`SolicitudAtencion`	Cliente, motivo, mensaje, prioridad, estado, respuesta, responsable, fecha de resolución.	Estados Pendiente, En Atención y Resuelto.
`Notificacion`, `Actividad`	Tipo, título, descripción, fecha y lectura.	Siete tipos de notificación y siete de actividad.
`AppConfig`, `UserProfile`	Ajustes y perfil del negocio.	Metas por defecto: 10 servicios y 1.200.000 de ingresos.

Modelo	Atributos relevantes	Comportamiento
`ChatMessage`	Texto, emisor, tipo de emisor, fecha.	Tipos: Cliente, Agente IA, Administrador y Sistema.

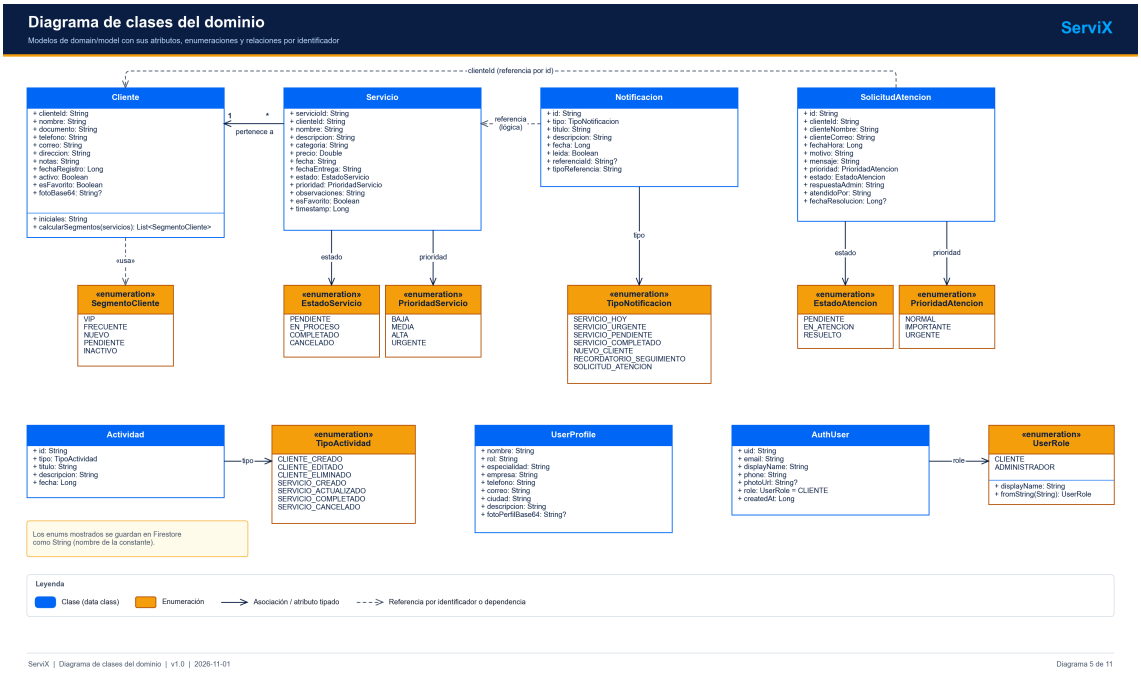


Figura 10. Diagrama de clases del dominio y los repositorios

Fuente: Diagrama del Equipo ServiX (04_Arquitectura_Diagramas).

5.3 Casos de uso del dominio

Tabla 19. Casos de uso agrupados

Grupo	Clases	Colaboran con
Autenticación	`SignInUseCase`, `SignUpUseCase`, `SignOutUseCase`, `SendPasswordResetUseCase`, `SignInAnonymouslyUseCase`	`AuthRepository`
Clientes	`AddClienteUseCase`, `UpdateClienteUseCase`, `DeleteClienteUseCase`, `RestaurarClienteUseCase`, `ToggleClienteFavoritoUseCase`	Clientes, actividad, notificaciones
Servicios	`AddServicioUseCase`, `UpdateServicioUseCase`, `UpdateServicioEstadoUseCase`, `DuplicarServicioUseCase`, `DeleteServicioUseCase`, `RestaurarServicioUseCase`, `ToggleServicioFavoritoUseCase`	Servicios, clientes, actividad, notificaciones, Make, configuración, Auth
Notificaciones	`MarcarNotificacionLeidaUseCase`, `MarcarTodasNotificacionesLeidasUseCase`, `DeleteNotificacionUseCase`	`NotificationRepository`
Atención	`CrearSolicitudAtencionUseCase`, `TomarSolicitudAtencionUseCase`, `ResponderSolicitudAtencionUseCase`, `ResolverSolicitudAtencionUseCase`	Atención, actividad, notificaciones

Grupo	Clases	Colaboran con
IA	`SendAiChatMessageUseCase`	`AiChatRepository`, `CrearSolicitudAtencionUseCase`
Configuración	`UpdateObjetivosUseCase`, `UpdateProfileUseCase`, `UpdateConfigUseCase`, `MarcarOnboardingVistoUseCase`, `ResetDataUseCase`	`ConfigRepository`

5.4 Datos

Tabla 20. Implementaciones de repositorio

Repositorio	Implementación Firebase	Ruta de Firestore	Listener
Autenticación	`FirebaseAuthRepository`	`usuarios/{uid}`	`AuthStateListener` de Firebase Auth
Clientes	`FirebaseClienteRepository`	`usuarios/{uid}/clientes`	`addSnapshotListener`
Servicios	`FirebaseServicioRepository`	`usuarios/{uid}/servicios`	`addSnapshotListener`
Notificaciones	`FirebaseNotificationRepository`	`usuarios/{uid}/notificaciones`	`addSnapshotListener`
Actividades	`FirebaseActividadRepository`	`usuarios/{uid}/actividades`	`addSnapshotListener`
Configuración	`FirebaseConfigRepository`	`usuarios/{uid}/configuracion/{app_config, user_profile}`	Dos escuchas de documento
Atención	`FirebaseAtencionRepository`	`solicitudes_atencion`	Escucha filtrada por `clienteId` para clientes; completa para administradores

Los repositorios obtienen el `uid` del usuario actual para componer la ruta; al cerrar sesión cancelan las escuchas. `FirebaseAtencionRepository` consulta el rol en `usuarios/{uid}` antes de decidir el filtro y se suscribe mediante `AuthStateListener` para no abrir la escucha antes de autenticarse (hallazgo H-19).

5.5 Presentación

La aplicación expone un único `MainViewModel` que concentra los flujos de estado (`clientes`, `servicios`, `actividades`, `notificaciones`, `userProfile`, `appConfig`, `solicitudes`, `chatMessages`) y las acciones de la interfaz. Las pantallas son funciones composables sin lógica de negocio. El archivo supera los 570 líneas y 24 KB, lo que se registra como deuda técnica.

Tabla 21. Pantallas y componentes de presentación

Elemento	Descripción
`AuthScreen`	Registro e inicio de sesión con selector de rol; atajos solo en depuración.
`HomeScreen`	Estadísticas, metas, atención requerida y búsqueda global.
`ClientesScreen`, `ClienteDetalleScreen`	Lista y detalle de clientes con segmentos.

Elemento	Descripción
`ServiciosScreen`, `ServicioDetalleScreen`	Lista, filtros y detalle con indicador de progreso.
`HistorialScreen`	Actividades y centro financiero.
`ChatScreen`	Conversación con el asistente y estado del escalamiento.
`ClientePortalScreen`	Portal del cliente.
`PerfilScreen`	Perfil, ajustes y prueba de conexión con Make.
Componentes y diálogos	Tarjetas, chips de estado y prioridad, estrella de favoritos, tutorial, formularios y centro de notificaciones.

6 Manejo de errores y resiliencia

Tabla 22. Estrategias de manejo de errores

Escenario	Comportamiento actual	Evaluación
Fallo de inicialización de Firebase	Se registra y se activan repositorios simulados.	Resiliente pero silencioso (H-11).
Credenciales inválidas	`AuthResult.Error` con mensaje del SDK o texto propio.	Adecuado.
No se puede leer el rol	Se asume `CLIENTE` (mínimo privilegio).	Adecuado.
Falla del modelo de IA	El repositorio devuelve `Result.failure`; el caso de uso muestra un mensaje genérico de inconveniente y sugiere reintentar o pedir un asesor.	Honesto: no simula una respuesta; el texto no distingue la causa.
Webhook sin URL	Se omite el evento con una advertencia en el registro.	Adecuado.
Fallo HTTP hacia Make	Se registra; la operación de negocio continúa. La prueba manual informa el código HTTP.	Adecuado.
Error de escucha de Firestore	Se registra y se conserva el último estado.	Mejorable: sin aviso al usuario.

Advertencia. Mocks silenciosos

La degradación a repositorios simulados evita cierres inesperados, pero oculta fallos de configuración. Debería mostrarse un aviso visible o bloquear la escritura cuando Firebase no está disponible.

7 Persistencia y reglas de seguridad

`firestore.rules` y `firebase.json` están versionados. Las reglas definen las funciones `isSignedIn`, `isOwner`, `isAdmin` (que lee el rol almacenado y acepta los valores heredados), `isValidRole` e `isShortString`. La tabla de la sección 3.7 resume los permisos por ruta; la explicación completa de cada regla se encuentra en DOC-07.

Operaciones de datos relevantes: el borrado de un cliente elimina también sus servicios consultando `servicios` por `clienteId`; la restauración vuelve a escribir cliente y servicios; marcar todas las notificaciones como leídas actualiza en lote las no leídas.

8 Integración con IA y con Make

8.1 Asistente de IA

El flujo completo se documenta en DOC-08. Desde el punto de vista arquitectónico, la integración se encapsula tras la interfaz `AiChatRepository`, lo que permite sustituir el proveedor y probar el caso de uso con un doble.

Tabla 23. Parámetros del asistente

Parámetro	Valor	Origen
Proveedor	Firebase AI Logic con Gemini Developer API	`GenerativeBackend.googleAI()`
Modelo	`gemini-2.5-flash`	`MODEL_NAME`
Temperatura	0,7	`generationConfig`
Máximo de tokens de salida	800	`generationConfig`
Historial enviado	Últimos 6 mensajes, iniciando por un turno de usuario	`MAX_HISTORY`
Instrucción de sistema	Contexto construido por `buildSystemContext`	`SendAiChatMessageUseCase`

8.2 Make

`OkHttpMakeWebhookRepository` implementa `MakeWebhookRepository` con dos operaciones: envío asíncrono (`sendServicioEvent`) y sincrónico con resultado (`sendServicioEventSync`, usado por la prueba de conexión y el reporte manual). El JSON incluye el evento, el servicio, los datos del cliente y del técnico y una marca de tiempo. Los eventos se disparan al crear, actualizar o completar un servicio.

> **Nota. Interfaces sin implementar**
> `RetellAIRepository` y `N8nWebhookRepository` están declaradas en el dominio pero no tienen implementación ni se usan. Se conservan como contratos para trabajo futuro y se registran como deuda (H-13).

9 Calidad: atributos y tácticas

Tabla 24. Atributos de calidad y tácticas aplicadas

Atributo	Táctica en ServiX	Evidencia	Brecha
Seguridad	Reglas de mínimo privilegio, rol inmutable, sin claves en el cliente, validación de longitudes.	`firestore.rules`; revisión de código.	Autoasignación de rol; App Check; sin pruebas de reglas.
Mantenibilidad	Capas, interfaces de repositorio, casos de uso, nombres descriptivos.	Estructura de paquetes.	`MainViewModel` extenso.
Testabilidad	Repositorios por interfaz y dobles; pruebas en JVM.	29 pruebas aprobadas.	Sin pruebas de repositorios Firebase ni de reglas.
Disponibilidad	Degradación a mock; tiempos de espera de red; escuchas con reintento del SDK.	Código.	Degradación silenciosa.
Rendimiento	Escuchas en tiempo real; historial y tokens acotados en la IA.	Código.	Sin mediciones.
Portabilidad	Wrapper de Gradle; compilación en Windows y Linux.	Compilación local y run de CI.	Solo Android.
Usabilidad	Material 3, modo oscuro, estados vacíos y de error.	Revisión de código.	Sin pruebas con usuarios ni en dispositivo.

10 Riesgos técnicos y deuda técnica

Tabla 25. Deuda técnica y riesgos de arquitectura

ID	Elemento	Descripción	Impacto	Acción propuesta
DT-01	`MainViewModel` grande	Concentra estado y acciones de toda la aplicación (más de 570 líneas).	Dificulta pruebas y cambios.	Dividir por funcionalidad: autenticación, clientes, servicios, atención y chat.
DT-02	Mocks silenciosos	La degradación a mock no avisa al usuario.	Pérdida percibida de datos.	Aviso visible o bloqueo de escritura (Bug 46).
DT-03	Interfaces Retell y n8n sin implementar	Contratos sin código.	Confusión sobre el alcance.	Implementar o eliminar (H-13).
DT-04	Registro con rol Administrador	El cliente elige el rol y las reglas lo aceptan.	Escalada de privilegios.	Asignar roles desde el servidor (Bug 45).
DT-05	App Check no inicializado	La dependencia existe sin proveedor configurado.	Uso de la IA fuera de la aplicación.	Registrar y activar el proveedor.
DT-06	Aislamiento de árboles por usuario	Los datos del cliente y del administrador están en árboles distintos; la notificación de una solicitud se escribe en el árbol del usuario activo.	El administrador puede no recibir la notificación de un cliente; pendiente de verificación en ejecución.	Escribir la notificación en un destino común o en el árbol del administrador.
DT-07	Dos paquetes de interfaz	`ui.screens.AuthScreen` fuera de `presentation`.	Inconsistencia menor.	Mover a `presentation.screens`.
DT-08	Datos demostrativos por defecto	`UserProfile` y mocks contienen datos ficticios de un negocio de ejemplo.	Datos de muestra visibles en cuentas nuevas.	Valores neutros por defecto.

Nota. Origen de DT-06
DT-06 es un hallazgo de análisis estático de `CrearSolicitudAtencionUseCase`, `FirebaseNotificationRepository` y `solicitarServicioCliente`. No se ha reproducido en ejecución y debe confirmarse con la prueba CP-041.

11 Evolución futura

Tabla 26. Evolución propuesta

Prioridad	Mejora	Beneficio
Alta	Asignar el rol administrador mediante claims desde un entorno de confianza.	Elimina la autoasignación de privilegios.
Alta	Pruebas en dispositivo y con el emulador de Firebase y de reglas.	Verifica el comportamiento real.
Alta	Registrar App Check y configurar cuotas de la IA.	Control de abuso y de costos.
Media	Dividir `MainViewModel` y adoptar un framework de inyección si el proyecto crece.	Mantenibilidad.
Media	Destino común para notificaciones de atención.	Entrega fiable al administrador.
Media	Fijar una imagen estable del agente de CI y añadir validación de build como política de rama.	Reproducibilidad.
Baja	Implementar o retirar Retell y n8n.	Claridad del alcance.
Baja	Notificaciones push (Firebase Cloud Messaging).	Mejor experiencia de atención.

Anexo A. Trazabilidad de decisiones con los requisitos

Tabla 27. Relación entre decisiones arquitectónicas y requisitos

Decisión	Requisitos que satisface
ADR-01 Firestore por uid	RNF-001, RNF-011, SEC-01, SEC-02
ADR-02 Firebase AI Logic	RF-018, RNF-002, SEC-06
ADR-03 Inyección manual	RNF-006, RNF-009
ADR-04 Roles inmutables	RF-002, RF-005, SEC-03
ADR-05 Mocks	RNF-009, RNF-017
ADR-06 Casos de uso	RNF-006; RF-006 a RF-023
ADR-07 Intención por palabras clave	RF-019
ADR-08 Webhook opcional	RF-022, RNF-016